Ctvrta prednaska

Tuesday 10 March 2026 14:06

Kontext procesu je casovy rez procesem -> podivame se na stavy procesu které pouziva a …...
JE TO STAV PROSTREDKU KTERÉ POUZIVA PROCES .. A URCUJE JAK BUDE POKRAČOVAT JEHO BĚH .. Stav souboru,socket, atd

Obecně-> co všechno mze ovlivni chovani procesu .. Ale to není mozne .. Tam spadají lidi ..
--- pak by to bylo vše co ovlivňuje člověka -- nakonec by byl kontext kazdeho procesu cely vesmir …
Je docela jasne proc se tam nepocitaji lidi, meteority, stromy atd..

1. OS je nespravuje nemuze je ovlivnovat a upravovat … jsme MOC POMALÝ ..

-- prostředky OS .. Jakmile pouzijeme prostredek stane se soucasti kontextu a muze to ovlivnovat ..
My musíme prostředky explicitne otevirat a uvolnovat .
JE vhodne uzavirat soubory ….. Pouziju prostredek -> ZAVŘU HO ..

DO KONTEXTU PROSTREDKU SPADAJI STAVY PROSTEDKU KTERÉ JSOU MOMENTALNE OTEVRENY:
VEŠKERA NAŠE PAMĚT MBAJTY
SOUBORY OTEVRENE
DISPLAY (PIXELY)

-- MULTITASKING -> DEFINICE ?

Pokud bychom řekli ze existuje okamzik ze bezi vice procesu … (blbost moc WEAK DEFINICE)

Cím jsou procesu na sobe a HW nezavyslejsi tim je to lepsi multitasking … není presna definice kdy to zacina byt muoltitasking ..

Hlavním předpokladem je schopnost: kontext switch ?

Vezmem proces ulozime ho na disk/operacni pamet .. Nekam .. A potom obnovime kontext jineho procsu a teprve pak ten proces/kontext zacne behzet
: jde o prepinani kontexnu ..

NEZAVISLOST PROCESU NA PROCESECH

JAKOU ČÁST KONTEXTU NEMUSIME VŮBECUKLADAT V PROCESU …

Kontext predstavit si jako stav bytu .. V okamziku kdy odchazime ukladame si stav bytu ?
Byt/pokoj je vlastnen namy a pouzivame ho do jiste miry jen my .. Když se vratime mel by vypadat uplne stejně …

Je to **VYHRAZENÉ .. NEMusíme ukládat kontext který je VLASTNĚNÝ PROSTŘEDKEM !**

Používá ho jen ten daný prostředek ..

Multitasking je založený na VIRTUALIZACI PROSTŘEDKŮ

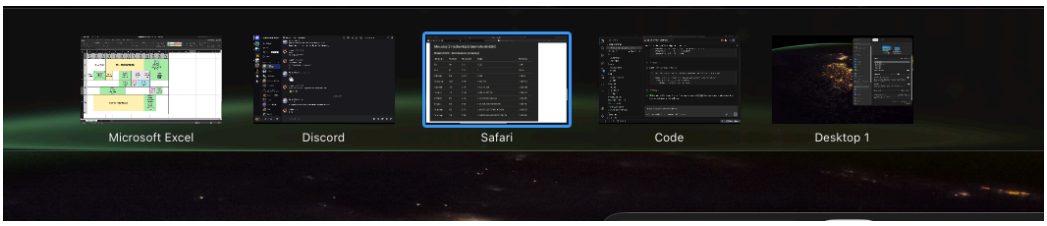
JAK SE VIRTUALIZUJE DISK ? Vytvari se na nem soubory -> soubor uz muze byt vyhrazený … vetsina souboru se kterymi pracuje nejaka aplikace uz jsou vyhrazené ..

Místo jednoho disku/svazku máme miliony souboru které jsou snadno sdilitelne

Jak se virtualizuje sitova karta ? Každý proces vlastní NICku s vlastnim kabelem FUUUJ
CASOVY/SOUBOROVY MULTIPLEX
Když pouzivam internet tak pouzivam nejakej tcpip atd a tam jsou pakety a ten paket je ten prosředek nas proces pouziva mnozinu paketu y ty pakety se stridaji na fyzicky lince a bezi paket další paket další paketa diky tpmu si každý proces mysli ze ma pro sebe nejaky socket .. Svůj vlastní.. Ikdyž je tam fyzicky vlastně jen jedno spojení …

Kde byl takovej ten problem když se začínaly telefony ? Každej musel byt propojený s každým XDD …
Byla nejaka ustředna kdy se to propojovalo a byl to nejakej jednoduche jzakladni multiplex … jedne člověk mohl zabrat celou linku .. Tim ze ses zrychluje multiplex .. Dostavame se do toho že několik procesů může používat jednu linku.

A jak se virtualizuej display ? ?
Jsou to ty okna které vidime



Ale není to samozřejme 100% virtualizace ..
Frame buffery by se nevešli do paměti jakoby ..
OS v pripade oken nedela virtualizaci uplne .. Stara se o menu zahlavi zapati .. O ten vnitrek se nestra .. On jen oznami ze se něco zmenilo a MY JSME NUCENI TO PREKRESLOVAT .. Není to pomoci frame bufferu ale pomocí .. Překreslovaní ma vyhodu ze muzeme rovnou chodit k GPU a je to RYCHLEJŠÍ

My to muzeme chapat že to co prekresluje je něco jako CANVAS .. My jsme odpovedni za prekresleni.. Ale nemusíme se starat o zadani toho co musíme překreslit ..

TO JE PROSTOROVÝ MULTIPLEX ..
Časový by byl takový, že by hodně rychle všechno blikalo XDDDD

Některé prostředky se špatně virtualizují -> přidělují (tiskárna)

Prostorový ani časový multiplex není reálný ..
Udělá se náhradní (tisková fronta) každej si do toho muze lest jak chce
Potom je jeden proces který ma k dispozici tiskovou frontu (tu substituci tiskarny) a PRISTUPUJE DÍKY TOMU JEN JEDEN PROCES K TOMU

Další hezká ukázka: zvuková karta … kdyby si kazdej proces posilal co chce tak by se prerusovaly atd xd

Ovlada ji jen jeden proces a nabidne nejaky mixer/audio .. Tam my se pripojime posilame zvukova data a nejak si to rozebere ..
ZVUKOVÝ MULTIPLEX .. Vic věci najednou a nevadi to .. Třeba zeslabi pipnuti když hraje hudba .. AudioMMIX

JEDNE POUZIVA ZKUVKOKU A VSEICHNI OSTATNÍ Pouzivaji zastupní zarizeni

Ale zajímavé je to že takhle zvladenem evirtualizovat téměř cokoliv .. To je to !!

Virtualizace procesoru ? Přes hypervizor nejak by to šlo ? Xd duvod je to ze by to bylo dost neefektivní ..
V procesoru musíme hlavne ukladat registry ..

Dneska to jsou spíš už kilobajty ..

V jádře má každý proces svůj zasobnik . Když zavolame sluzbu jadra tak vstoupi do jadra a vse si ukalda na zasobkniky .. Roste roste roste roste .. Jak postpupe vstupujeme do dalsich a dalsivh rutin rozhodeneme se prepnout kontext .. Vezemem ukazatel na vrcholu zasobniku .. SP (stack pointer) a pressuneme ho na jinej zasobnik.. A ještě predtim zavol tady push A nebo jinou isntrukci ta nam ulozi všechny registry a v okamziku když ma všechny registry ulozene presune je na jiny zasobkik kde jsou zase nejake rehistry ulozene a ten zase zavola pop A precte kontexty ze zsasobniku a v tom zase začně běžet jako jiny proces

Cele to je ulozeni kontextu na zasobnik a potom prepnuti tooh kontextnu na jinej zaosbnik a obnoveni registru a pak se dostaneme do .. .kontextu procesu ? ?... .
předtava proces vstoupi .. Když něco vykonava nechava po sobe stopy .. Tim s iuklada svůj stav potom staci jen zmenit vrchol zasobniku pak ten proces zacne pouzivat jiné hodnoty ma jinak ulozene navratove adresy atd atd

VIME CO JE ONTEXT VIME CO JE PREPNUTI KONTEXTU A TY VĚCI SE STAVAMA A ZASOBNIKEM A PREPINAIM KONTEXTU ..

DRUHY multitaaskingu ? Finally ?

Prepinani procesu ?

MS dos -> start ulozi se to co je v procesu/kontextu ? ?stav asi .. A novy proes se nastaruje jako kdyby byl novej a procesu vznikne novy kontext … když ho ukoncime tak se neuklada stav ale jen kontext .. A my cekame nez skončí ..

Když jsme delali něco v txt editoru tak jsme stopnuly editor ukoncili ho ulozili proces zapnuly kalkulačku a spocitali.. Pak ukoncili kalkulačku a pak znovu zapnuly txt editor … dobré bxylo, že preponuti proecsu bylo presne na nejakem miste .. Třeba gui nebylo soucasti procesu který se ukladal .. Neukladal se stav display .. První se ulozil stav grafiky a pak az zavolat kalkulačku .. A to jsme mohli delat my protože jsme vedeli kdy se stane prepnuti kontextbu klidne tom ohlo trva CELOU SEKUNDU …

TOMUHLE SE MULTITASKING JEŠTĚ RIKAT NEDA PRVNÍ WIN 3.11 TAKY BEZEL NA TOMHLE..ikdyz o tom tvrdili ze to byl multitasing

První se nazyval KOOPERATIVNÍ (ikdyz to taky nebyl xddd) dela to za nas jadro operacniho systemu samo.. Při kazdem dlouhodobém volání se to deje … co to jedkouhodobe ? Cekame na nejakej proces ..

Get char -> vstoupi do OS a cekame az někdo zmackne klavesu aby to vstoupilo .. Procesor odpocival xddd

Při dlouhych volanich muzeme prave prepinat kontext a jakmile zavolame dlouhou sluzbu (ceka na data synchr udalost atd.. Z hlediska procesuru to trva několik tisíc instrukci dlouhodobe)) a aby mel procesor co provadet tak zavolal jiny proces a PREPNUL KONTEXT .. To mělo vyhodu

Z heldisksa uzivatelu to vpadalo jako multitasking … nemusime vse virtualizovat . Vime o prepnuti kontextu ? ..
Muzeme proste před kazdym get char ulozit stav prostredku xdd .. Zatím mame stále moznost ridit si stavy svyhc prostredku a diyk tomu ta virtualizace nmusí být 100%ntní ..

Staci se podivat zda ma proces něco ve fronte udalosti .. Pokud ano ?uz jsem zapomněl :DDDD

Cekani je prirozene . .. Dokonce k tomu existuje sytl programovani: UDALOSTNE RESENE PROGRAMOVANI
Je s tim kompatibilní :D D

Problem je ze se nam program rozpadne do izolovanych funkci které se mezi sebou nevolají xddd
As\nchroni programovani to resi lip co idk ?

Vyhoda kooperatovniho multi .. Muzeme drtet sluzbu)asi proces myslle) jak dlouho se nam zachce
Klasick prikaz NEKONEČNÁ SMYČKA ..
Udelame vypocet který ma 10 vterin .. A nikdo jiny k tomu ted nemá pristup LOL

PROBLEM JE DOBA ODEZVY - musime dodržovat nejake minimalni casy

Ještě před asynchrnonnim jsme to museli proste rozsekat ..
Hlavni problem je privlastneni procesoru

CO se stane když OS zjisti ze musí odebrat procesor ale zadny z jinych procesu nemá nic ve fronte udalosti ?
Bezny Procesor se neda zastavit .. EXISTUJE IDLE PROCES -- půjčí si ho a pak se ho vzdá ..
Když mame 16 jader vetsina z nich je v idle procesu pac je tezke je zatizit xd

PREDIKTIVNI (ne je to nejak preaktivni nebo tak něco spatne jsem slyšel)
MULTITASKING - kooperativní + zachrana brzda
Necha kooperovat dokud to jde a pak mame brzdu

CO JE TO TA BRZDA ? -> když se vracime ze sluzby kazde i z dlouhe i z kratke ..
IKdyž zavoleme kratkou dojde k prepnuti .. ?
Při kazdem navratu z přerušení muze dojit k přepnutí kontextu

1. Softwarové přerušení (třesnička na dortu kratka sluzby)
2. Exceptions (chyba v OS .. Taky tresnicka na dortu vetsina procesu to neudela)
3. HW preruseni .. Nejdulezitejsi -> jsou vyvolana mimo ten procser který nad nimi nemá kontrolu .. Prejde do jadra a při navratu z prerušeni muze dojit k prepnuti kontextu

Každý distributor SW musí udelat ovladače které se musí virtualizovat a musí byt v jádře .. Takze další sluzba která to zvetsuje ?

Udelam preaktivni na jednodchem zarizeni je jednoduceh
Udelat to na nejakem vetsim OS je vetsi orisek ..

Celé je to zaručeno tim ze musime mit BEZPEČNÝ ZDROJ PŘERUČENÍ .. -- bez přerušení to nejede a NEPŘEPÍNÁ

- Nejčastěji je to časovač mimo OS .. Timer který to prerusuje třeba kazdou milisekundu .. Je tam hlavne pro čas .. Ale když mame timer tak mame zaručení, že diky timeru jsme schopni prepinat kazdou milisekundu …

Dneska jsou jiné zdroje presueni .. Napr sitova karta (pakety) SSd disky .. Atd .. Timer je opravdu jen takova failsafeBREAK xd

Jak se chova když je malo zatizen -- ve skutecnosti bezi kooperativne a při to bezi ještě IDLE
Bezi proces P1 -> K (kooperetvni stav procesu) -> bezi IDLE taky se vzda -> K -> IDL -> K -> P1 ->

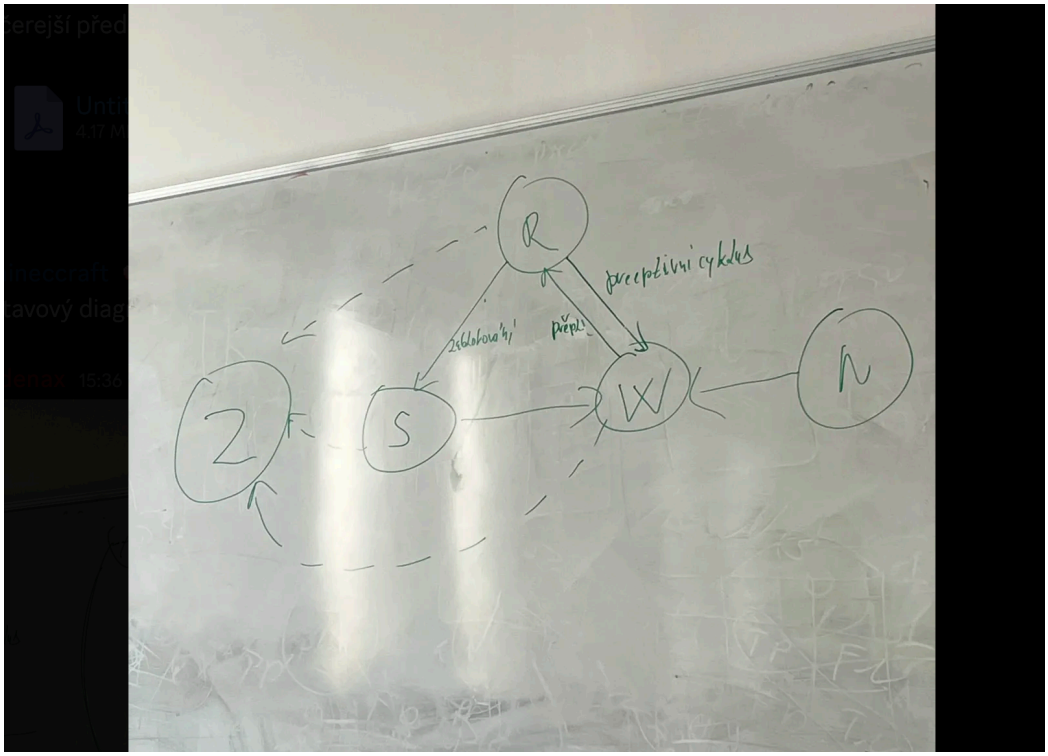
Běží proces P1 -> K (kooperetivní stav procesu) –> běží IDLE taky se vzda –> K - >IDL –>K –> P1–> P–>P2–>K–>IDL
Když je časově .. hodne zahulceny mame tam přerušení

ČASOVÁ VARIANTA ::
Nevidim na obrazek
Řekl ze P3 se klidne vzda procesoru xd SOUTĚŽÍ O PROCESOR .. Ale je to vzacne .. Pouze při vysokém zatížení

A TED TA NEJDULEŽITEJSI VĚC NAŠÍ PREDNASKY DNESNI -> tohle tam určitě bude pokud si zvolime statice od nej otazku

JAK PROCES VNIMA SVŮJ ŽIVOT ?
Nazvoslovy je fišerovske

RUNNING - R
SLEEPING - S
WAITING - W



W - ma co delat byl MU ODEBRAT PROCESOR jde to –> Running …
Když mu něco s procesorem (NEDOBROVLNE ODEBRAN) .. Tak jde zpatky do waiting
TIM VZNIKA PRVNÍ CYKLUS: preeptivní cyklus (naplánování R–>W
W—— > Reschedulong

z N NEW SE DOSTANEME DO WAITING a z R nebo W do Z (**Zombie)**

Cyklus mezi running –> sleeping je zablokovani

NEW –> ON SI NEJDŘIVE musí vyhradit místo v tabulce procesu a dat stav N new (EXISTUJE ALEN NI VYTVOŘEN)
Druhá fáze : MUSÍME HO VYTVOŘIT .. Vytvori se tak jako kdyby uz existovali ale byli v pozastaveny v okamziku startovniho bodu
Celé je to o emulaci toho že je pauznutej a spusti se. Ve startovnim bode

2 pristupy vytvareni emulace:

1. WIDLE -> Proces musí vse udelat za sve dite (prideli pamet tabulku stranek něco vytvorit, namapovat data, pirpavit zasobniky, user spaec, kernel jak kdyby bezel, napodobit stavy registru atd .. Vytvari virtualni novej proces a zbytek věcí JAKO KDYBY BĚŽEL .. –>> **JE TO DOST POMALE NEVYPLACI SE VE WINDOWS VYTVARET PROCESY ČASTĚJI**
   MÍSTO TOHO SE VE WIN POUZIVAJI VLAKNA

2. UNIX -> FORKOVÁNÍ . Proces se zduplikuje do sve nove kpie .. Zavolame v procesu fork v jadru se udela duplikat.. A pak se vratime do formku ve 2 procesech .. Jednou do puvodniho RODIČ a poté do dítěte(JAKO KOPIE RODIČE)
   Pokud chce pouzivat něco jineho nez rodic musí ZAHODIT JEHO VĚCI .. A navrhne si vlastní prostředky ..
   v unixu se muze rozhodnout zda bude pouzivat prostředky rodiče nebo chce bezet podle jineho programu a OPUSTIT DOMOV !!
   Rodič ma chvili rizeni nad svým ditetem priparvit mu to a pak spustit .. WIN JE SLOZITEJ

   Jedna mala nevyhoda .. Jak se provadi duplikovani ? Koodvy region se nemusi duplikovat .. Je read only takze se nasdili … datovy se nesmi .. ?? Co něco s copy write .. Když dite něco zapise tak se to kopiruje .. Když nezapise nic nic nekopirujeme ??? Idk .. Je to HROZNĚ RYCHLÉÉÉ !!
   Zasobnik se taky usí udělat COPY WRITE

   Čím se lisi dite od rodiče ? Nějakej fork nebo co .. (se spešl nazvem if fork myslim).. .. U rodice vraci 5 ditete a dite ma 0 …

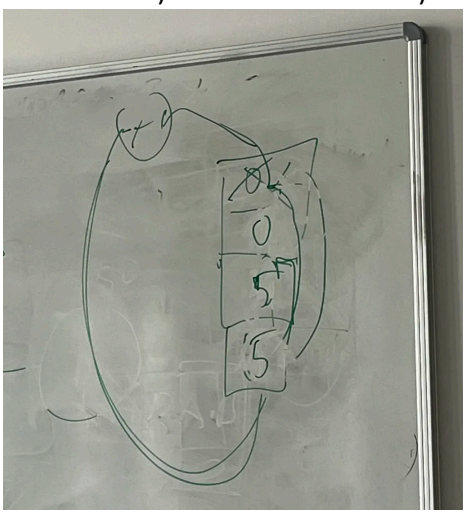
   JAKMILE POCHOPIME FORK POCHOPIME JÁDRO UNIXU

   Jednou zavolame dvarkat vratime bezi 2 nezavisly toky rizeni a když je to konecne hotovy tim to presune rodic do stavu sleeping … a není mezi nimi uz nejaky prisli explicitní stav.. K rodici se vratime az na zombie ??
   ¨

   A TED SE PRESOUVAME DO STAVU N–> WAITIING
   : Fronty ve kterých se předbíhá podle PRIORIT
   PRIORITNÍ FRONTY
   Dneska to je jinak ale stále to vypada podobně
   PŘÍKLAD PRIORITNÍ FRONTY: jidelna na zš .. Predbihaji nas ucitelé, vyssi rocniky atd ..
   Kdo mas nižsi prioritu aotuomatickyi ous predbehnuti .. A zaradi se za stejné/vyšší

Kde mas nizsi prioritu automaticky jas predbehnuti .. A zaradi se za stejne/ vyssi

Vždy si on vezme proesor na vrcholu vezme pouzije a zařadí na konec ..
Vetsinou jsou prioriti inverzní (čím mensi cislo tim vetsi priorita)
Když ma třeba prioritu 0 tak se napkanuje udela da se zpatky a zase se naplanuje pac ma porad prioritu ? Lol
NENAŽRANY UCITEL ZAKLADNI SKOLY : DOSTANE JIDLO PREDBEHNE DOSTANE JIDLO PREDHBEHNE..
Statická by nešla.. .PROTO MÁME DYNAMICKÉ FRONTY.
Vetsinou se kazdopadne kombinuje staticka část + DYNAMICKA PRIORITA
SNIŽUJEME)TAKZE ZLEPSUJEME) PRIORITU kazdym cyklusem těm kteří čekají …



Když se ten s 5 dostane na 0 tak se ten co ma proad nulu dostane za nej (zařadí se za stejnou nebo vyssi prioritu)

Ten s petkou co ma ted 0 se dostane na řadu a jde na konec fronty opet s puvodnim .. Což byla pětka .. Atd

Running to je jasne : ma 2 přepinače .. KERNEL A NĚCO

SLEEPING -- ČAS A ? Něco …

Ve stav usleep 2 reprezentace ….

1. Normalni fronta.. Probudi se při udalosti ta první v te fronte
2. LEPŠÍ ŘEŠENÍ … –> **SPECIALNÍ FÍŠA POJEM** -> EUROCITY (NEBO EROCITY) TEN CO JE VE VLAKU NEJRYCHLEJSI ZVEDNE SE A OBSADI ZACHOD .. Zachod ve vlaku ..

   Jeden to stihne a ostani se taky probudi ale zjisti ze udalost uz byla a nestihli to … Ten nejrychlejší jede

ZOMBIE - DALŠÍ FÍŠA POJEM

1. SEREVRAŽDA .. proces zavola sluzbu, vrati se a presune se do zombie

1. SEBEVRAZDA - procse zavola sluzbu nevrati se a presune se do zombie
2. Vražda - linux KILL - někdo nekoho zavraždí
3. Smrtelný -> přistoupi nekam kam nemá tak DOSTANE BANÁNA -- z jadra uz nevystoupi ..

V lidskem zivote neexistují .. Ale tady každý proces končí ve stavu zombie ..

1. Proces sam sebe nemuze ukončit .. Odstranit se z tabulky procesů .. Nene to nejde bude z tebe zombík
   Mohl by bezet proces který neexistuje ..ale mezi dobou co smaze a nez se vypne .. Tak kdyby bylo preruseni zjisti s e ze bezi něco co neexistuje a zaroven ma prostředky které patri neexistujicimu procesu

   ZOMBIE VLASTNÍ MINIMALNI PROSTŘEDKY KTEROU JSOU MU ROZKRADENY ..

   Musí existovat proces který zombie proces zabjie ..
   JE TO RODIČ
   Zombie si nese info jak skončil .. A diky tomu rodič pozna zda a pokud ho zabít …
   Většinou rodič vola sluzbu wait .. Ceka az budou deti zombici a pak je odstranime .. Az kdy odstranime posledniho zombika tak pak až je odstranime …

   EXISTUJÍ BOHUŽEL RODIČE, KTEŘÍ SE O DĚTI NESTARAJÍ - můžou vzniknout dlouhodobě běžící zombíci …
   TO JE CHYBA !!

   Vadí nám nebo nevadí ? Obsazují tabulku procesů takže ANO VADÍ ..

   Není snadné zabít zombie .. KILLEM to nejde - zabijte JEHO RODIČE - a postara se o to funkce která se stara o sirotky ..

   Jak vytvorit zombie ? Rodic vytvari kolik zombie kolik potrebujeme (1 cyklus jedne zombie)
   TO JE LINEARNI

   Exponencionalni zombie bomba .. Kazde dite vytvori další zombie atd ….

   Preplni se celkova tabulka a zmrzne systém JEDNA Z MNOŽNOSTI DDOS (vic uzivatelu).
   ZMRZNE POCITAC
   -- jediny co tak root ma rezervu ….